

Honors Class Activity: Faking Randomness

Team Members: _____

Learning Objectives

- Recognize that a hypothesis test requires choosing a *test statistic*, and that two statistics measuring the same thing can differ enormously in how well they work
- Build a null distribution by simulation and compute an empirical p-value
- See how the amount of data needed depends on the design of the study, not only on the size of the effect

Part 1: Write a Sequence That Fools Your Instructor

Before every Eagles game, an official flips a coin at midfield. Suppose you had to fake that result — not once, but a hundred times, convincingly.

Close your laptop. Each team member takes a blank card and writes a sequence of **100** H's and T's on it. Your goal is to produce a sequence nobody could tell apart from 100 real coin tosses. Do not flip anything. Do not use a random number generator. Do not compare notes with your teammates. You have five minutes.

When you are done, your instructor will give each of you a second card holding a genuine sequence of 100 tosses, generated last night. Shuffle your two cards, label them **1** and **2**, and write below which is which. Then hand both cards in.

My ID code: _____ My fabricated sequence is card: _____

Before the reveal: out of ten pairs, how many do you predict your instructor will sort correctly? Write your number, and one sentence saying why.

Part 2: The Reveal

Your instructor's record: _____ correct out of _____.

What do you think your instructor looked at? Name something that could actually be computed from a sequence of H's and T's.

Part 3: Two Ways to Measure the Same Thing

A **run** is a maximal block of identical outcomes. The sequence **HHTTTHT** contains four runs — **HH**, **TTT**, **H**, and **T**. Its longest run has length 3. It also contains three **switches**: places where the sequence changes character.

For a sequence of n tosses there are $n - 1$ adjacent pairs, and the number of switches counts how many of those pairs disagree. Notice that the number of switches is always one less than the number of runs, so these two quantities carry the same information.

Open a fresh Jupyter notebook and enter the following.

```
import numpy as np
from datascience import *
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('ggplot')

def switches(seq):
    """Count how many adjacent pairs disagree."""
    count = 0
    for i in np.arange(len(seq) - 1):
        if seq[i] != seq[i+1]:
            count = count + 1
    return count

def longest_run(seq):
    """Length of the longest block of identical outcomes."""
    best = 1
    current = 1
    for i in np.arange(1, len(seq)):
        if seq[i] == seq[i-1]:
            current = current + 1
        else:
            current = 1
        if current > best:
            best = current
    return best

switches('HHTTTHT'), longest_run('HHTTTHT')
```

Now generate a genuine sequence and look at it. **Run this cell several times.**

```
toss = np.random.choice(['H', 'T'], 100)
print(''.join(toss))
print('switches:', switches(toss), ' longest run:', longest_run(toss))
```

How long is the longest run in a typical genuine sequence of 100 tosses? Is it longer or shorter than you expected?

Compute both statistics for both of your own cards and write them here. Then post your switch counts on the board.

	Fabricated card	Genuine card
Longest run		

Switches		
----------	--	--

Under the null hypothesis that a sequence is 100 independent fair tosses, what is the expected number of switches? Work it out — do not guess. What is the chance that any one adjacent pair disagrees, and how many such pairs are there?

Both the longest run and the switch count detect the same behavior. One is a much better test statistic than the other. Which one, and why? Argue in terms of how much of the sequence each statistic actually uses.

Part 4: Testing the Whole Class at Once

No single card gives strong evidence. Pooled together, the class does.

Enter the switch counts from the board.

```
# Replace these with the numbers from the board
fabricated = make_array(61, 58, 64, 55, 60, 57, 66, 59, 62, 54)
genuine    = make_array(48, 53, 44, 51, 49, 56, 47, 50, 45, 52)

print('students:', len(fabricated))
print('mean switches, fabricated:', np.mean(fabricated))
print('mean switches, genuine: ', np.mean(genuine))
```

Our test statistic is the **class total** number of switches in the fabricated sequences. To find out what that total looks like when nobody is faking, simulate an entire class of honest coin-tossers, many times over.

```
n_students = len(fabricated)
observed_total = sum(fabricated)

def simulate_class_total():
    total = 0
    for student in np.arange(n_students):
        toss = np.random.choice(['H', 'T'], 100)
        total = total + switches(toss)
    return total

simulated = make_array()
for i in np.arange(1000):
    simulated = np.append(simulated, simulate_class_total())

results = Table().with_column('Simulated class total', simulated)
results.hist(bins=25)
plt.scatter(observed_total, 0, color='red', s=100, zorder=3);
```

```
p_value = np.count_nonzero(simulated >= observed_total) / len(simulated)
print('observed:', observed_total, ' null mean:',
      np.mean(simulated).round(1))
print('p-value:', p_value)
```

Write the conclusion of this test in one sentence, in plain English. Say what the null hypothesis was and what the p-value does and does not tell you.

The genuine sequences were not part of the test. Why did we compute their mean anyway? What would you have concluded if that mean had come out to 62?

Part 5: How Much Data Would It Take?

The class result was overwhelming. But suppose you had to catch **one** student, working from that one person's sequence alone. How long would the sequence need to be?

First estimate how often the class switched.

```
q = np.mean(fabricated) / 99
q
```

Now simulate. For a sequence of length `n`, build a null distribution, then count how often a fabricated sequence would be flagged at the 5% level.

```
def detection_rate(n, q, trials=500):
    # What does the null look like at this length?
    null_counts = make_array()
    for i in np.arange(trials):
        toss = np.random.choice(['H', 'T'], n)
        null_counts = np.append(null_counts, switches(toss))

    # How often would a fabricator get caught?
    flagged = 0
    for i in np.arange(trials):
        fake = np.random.choice([0, 1], n, p=[1 - q, q])  # 1 means
        "switched"
        observed = sum(fake[1:])
        p = np.count_nonzero(null_counts >= observed) / len(null_counts)
        if p <= 0.05:
            flagged = flagged + 1
    return flagged / trials

for n in [30, 50, 100, 200, 300]:
    print(n, 'tosses ->', detection_rate(n, q))
```

About how many tosses would you need before a single fabricated sequence gets caught 80% of the time?

Add up all the tosses the class produced together. Compare that with your answer above. The behavior is identical and the statistic is identical, so why does one design need so much less data per person than the other?

Discussion Questions

Your instructor tried ten pairs and reported the results of those ten.

There were roughly twenty-five pairs in the stack, and your instructor handled all of them before choosing which ten to attempt. What is wrong with treating that hit rate as an honest estimate of skill, and in which direction is it biased? Where else in science does this same problem appear?

Suppose a student objects: “My fabricated sequence had a run of six heads in it, so your test is wrong about me.”

Is the student right? What is a hypothesis test actually a claim about — the individual case, or something else?

We chose a one-sided test: we only looked for *too many* switches.

Was that legitimate? What would have made it illegitimate? Suppose the class’s fabricated sequences had come out with *too few* switches — what should we have done, and what should we not have done?

In 1985, three psychologists analyzed thousands of basketball shots using runs — the very statistic you used today — and concluded that the “hot hand” does not exist. The finding became famous and stood for three decades. In 2018, two economists showed that the runs calculation was subtly biased against finding a hot hand, and that correcting it restored the effect. No new data were collected.

What does this episode suggest about the difference between a result being *replicated* and a result being *correct*? If the flaw was in the method rather than the data, why did thirty years of further study fail to catch it?